

Traveler DNA – Functional Description

Initialization & Update feeds

Daily Batch

and

Live feed *(only for customers implemented before 2023)*

Index

1	Introduction	5
1.1	Disclaimer	5
2	Overview	6
3	Solution introduction	7
3.1	Context	7
3.2	Scope	7
4	Solution description	9
4.1	Initialization of airline's system	9
4.1.1	Objective	9
4.1.2	Pre-conditions	9
4.1.3	Description	9
4.1.4	Output	9
4.2	Update of airline's system	10
4.2.1	Objective	10
4.2.2	Pre-conditions	10
4.2.3	Description	10
4.2.4	Output	10
4.3	Grammar	11
4.3.1	Context part	12
4.3.2	Dictionary part and source id handling	13
4.3.3	Customer part	15
4.4	Feed content	16
4.4.1	Profile filtering	16
5	Feed distribution channels	17
5.1	Batch (included in baseline)	17
5.2	Live feed (Add-on only for customers implemented before 2023)	17

6	Feed consumption guidelines	18
6.1	Profile dynamicity	18
6.1.1	Merge.....	18
6.1.2	Split	20
6.1.3	Rehook	21
6.2	Profile identifier	23
6.3	Which data to record?	23
6.3.1	Source references	23
6.3.2	Golden profile	26
6.3.3	Origin and Destination	28
6.3.4	Metrics	30
6.3.5	Making sure to record the latest version	31
7	Appendix	34
8	Glossary.....	35

Document control

Security level	CONFIDENTIAL RESTRICTED			
Company	Amadeus IT Group SA			
Department	TEC-ADT-T4A-CPI TRU-ALT-CPD-PPM-PER			
Author	Papa GUEYE Julien GEVREY Arnaud GENEST Gaelle QUERELLOU			
Reviewed by	Diego FERRERO Quentin AUFROY Veronica MILANI			13/10/2021
Approved by	Serge CORBUCCI			13/10/2020
Version	Date	Change	Comment	By
0.1	28/07/2020	Document creation		Gaelle QUERELLOU
0.2	22/07/2020	Update following R&D review		Gaelle QUERELLOU
0.3	23/07/2020	Update following Implementation		Gaelle QUERELLOU
1.0	12/10/2020	Document validation		Gaelle QUERELLOU
2.0	17/06/2022	Added Profile versions		Christophe GOMEZ
3.0	17/11/2022	Document alignment with xsd		Thibaud LUND PAULINE VITET Gaelle QUERELLOU
3.1	12/04/2023	Document update		Gaelle QUERELLOU

TEC-ADT-T4A-CPI / TRU-ALT-CPD-PPM-PER

1 Introduction

This document provides a Functional Description of the initialization and update feeds, that is aiming to synchronize data from Traveler DNA (formerly Customer Experience Management (CEM)) with an airline system.

1.1 Disclaimer

All screenshots, examples and resource locators presented in this document are not contractual. The final user interface may present some differences, but it will provide the same level of functionality.

2 Overview

Personalization of offers and services allows to improve customer experience, which is important in the travel industry. Personalization solutions for airlines need to provide a “single source of truth” about customers by consolidating customer data from various sources into unique customer profiles.

Customer airlines seek to integrate different data sources (PNRs, Tickets, Loyalty Profiles, communications, etc.) within Amadeus Traveler DNA (T-DNA) system in order to deliver relevant personalized servicing and travel experience at various customer touchpoints. Once consolidated in T-DNA and additional insight computed (e.g., metrics), those data can be synchronized back to the airline, to keep up to date its customer-related data bases.

This outbound feed from T-DNA is then a two-step process: an ***Initialization feed***, followed by an ***Update feed***:

1. **Initialization feed:** airlines use this export to create a copy of the T-DNA database on their side
2. **Update feed:** airlines use this export to ensure that the customer information is kept up to date

Amadeus team delivers this Solution Overview Document in order to ease T-DNA integration with other systems, and to provide a description of the outbound feed solution.

3 Solution introduction

3.1 Context

The replication of T-DNA data within airline's Data Warehouse can bring value in terms of analytics and reporting. In addition, the setup of an outbound live feed from T-DNA can be leveraged for transactional actions on the airline's side.

3.2 Scope

This document describes a solution allowing the replication of T-DNA data from Amadeus to airlines servers. With the feed, airlines can create an aggregated database containing the information from different sources (PNR, Tickets, Loyalty profiles, communications, etc.) associated to a customer.

Important warning: *The internal Amadeus identifier of the profile is the T-DNA customer id. We recommend the airline **not** to use this identifier as it 'volatile' and subject to change.*

The airline's database is initialized during a one-off process described in this document. Then, an outbound feed from Amadeus pushes the changes on any customer profile. These changes are done either via a **Batch** or via a **Live feed (only for customers implemented before 2023)**. For customers implemented starting 2023, the alternative of the Live feed is the T-DNA feed to DIH Data Hub.

PNR updates (at the end of transaction time), ticket issuances, etc. are examples of trigger types that activate the following processes:

- A new customer profile is created in the T-DNA database
- A customer profile is deleted from the T-DNA database
- A customer profile is updated from T-DNA database

Upon completion of these processes, an update is pushed to the airline with the corresponding modifications on the customer profile.

The customer profile format contains the following elements:

- "Identity": contains the identity elements of the customer
- "Contact": contains the contact information
- "Products": contains the Origin&Destination of the flight
- "Memberships": contains the loyalty information
- "Subscriptions": contains the subscriptions

- “Accounts”: contains the third-party accounts
- “Touchpoints”: contains the information related to a specific device.
- “DisruptionInformation”: contains the information of disruptions experienced by the passenger
- “Metrics”: contains the updated metrics of the customer
- “Tags”: enables API users to add one or several tags to a profile (e.g. VIP)
- “Communications”: contains the communications related to Alert feedbacks, etc.
- “Links”: Allows the API user to specify a link between two customer images. Eg: Linking a Communication to a specific PNR
- “AlertInstructions”: Identifier of the Alert Instruction
- “MergeHistory”: Specifies the TDNA id of the profile that has been merged
- “Version”: Provides the version number associated to the customer image

For a complete description of elements, please refer to the **XSD file referred in Appendix**.

New elements require additional development that need to be assessed and planned.

The following information are not provided in the scope of the initialization and update feeds:

- The segments to which the customer belongs

Notes:

- Amadeus does not keep the history of the changes that occurred to the customers
- The communication channels set up (SFTP or MQ for instance) is out of the scope of this document
- Next bullets are specific to Live feed (*) distribution channel (cf. [Live feed](#) section):
 - There is no retry mechanism for the update feed messages. Issues during distribution of the updates can be recovered but an action is required on Amadeus side.
 - The order of the messages is not guaranteed, the timestamp should be used in order to determine if the customer needs to be updated. Details on the general logic to apply are provided in section [Making sure to record the latest version](#)

(*) also named ‘Online feed’

4 Solution description

4.1 Initialization of airline's system

4.1.1 Objective

The objective is to initialize the airline data warehouse with the T-DNA database.

4.1.2 Pre-conditions

JSON interface between T-DNA and airline system is defined and agreed.

The airline manually requests the T-DNA database to export.

Communication channel is agreed and set-up.

4.1.3 Description

T-DNA manually triggers the full export of the T-DNA database into one or several zipped JSON files.

4.1.4 Output

T-DNA sends the JSON files to the airline via the agreed distribution channel (SFTP or MQ). Inside a file, each line corresponds to a customer.



Limitations:

- The full export is only run during the initialization and is not repeated afterwards

Exceptional conditions such as database crash may require a re-initialization of airline's database. Those situations are handled on a case-by-case basis.

4.2 Update of airline's system

4.2.1 Objective

Notify the airline's system on activities regarding its customer database and provide the updated information.

4.2.2 Pre-conditions

A customer is created or updated in the T-DNA database.
The airline can receive updates from T-DNA via an agreed channel.

4.2.3 Description

Upon update of a customer, T-DNA sends the new full image of the customer profile to the airline.

Note:

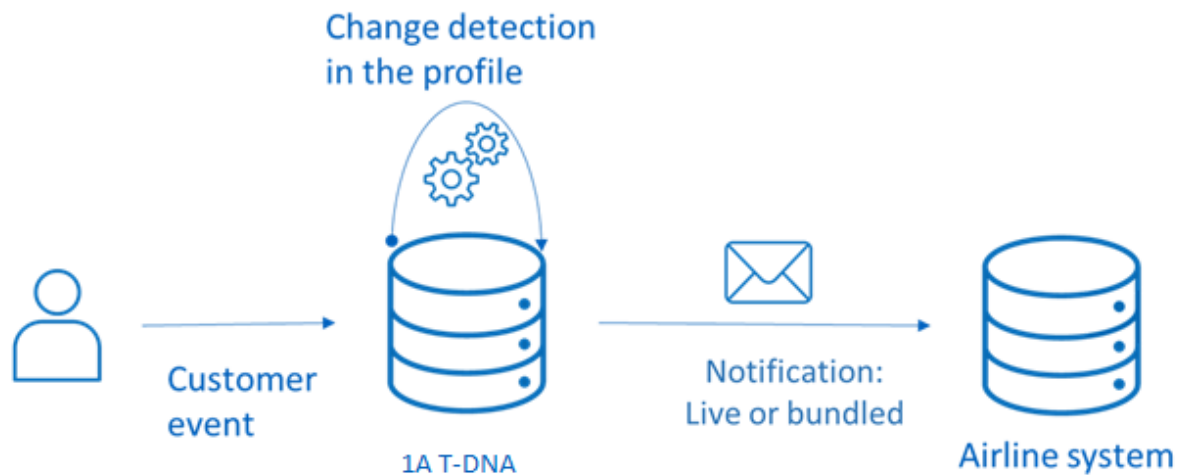
A change in metrics computed by Amadeus is not considered as an event for customer update and therefore does not trigger an Update Feed.

Also, a full synchronization of the metrics in a batch mode is outside of the scope of this document as it not part of the Update Feed product. However, refreshed metrics are provided to the airline system whenever there is a customer update.

4.2.4 Output

T-DNA sends the output to the airline via the agreed distribution channel (SFTP for the Batch or MQ for the Live feed):

- For the **Batch**, the output file contains one line per profile, each profile (or line) being represented in a JSON format
- For the **Live feed** (only for customers implemented before 2023), each time a profile is impacted, a JSON file is sent to the airline system in a queue

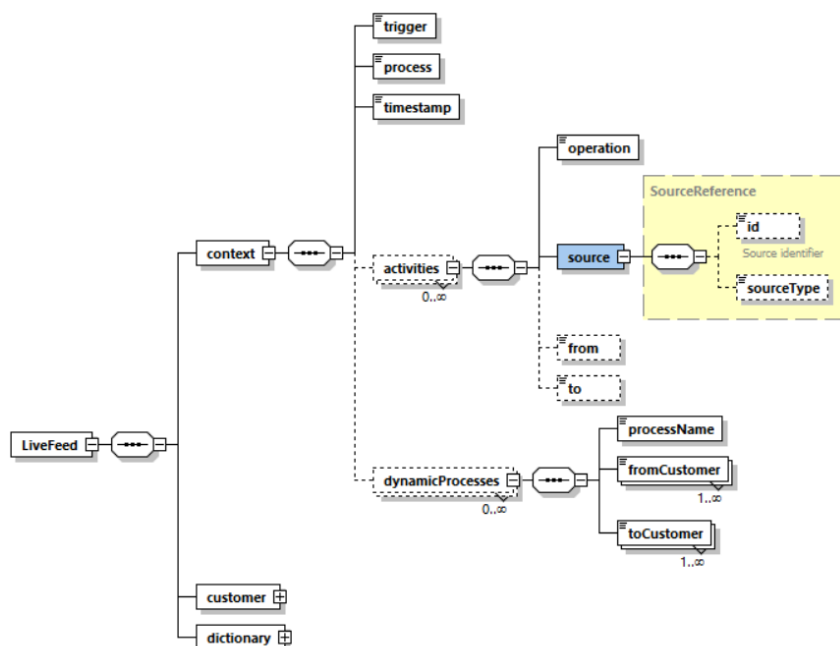


The airline receives the full new image of the customer.

Please refer to next sections for more details on the update's content.

4.3 Grammar

Both Initialization and Update feeds have a common grammar. The JSON object of each profile has the following structure:



amadeus

It contains three main parts detailed in the next sections:

- context
- customer
- dictionary

4.3.1 Context part

The context includes the trigger, process, and timestamp of the customer update.

Example:

```
{
  "LiveFeed": {
    "context": {
      "trigger": "PNR",
      "process": "CustomerUpdate",
      "timestamp": "2022-02-11T18:22:27Z",
    },
    "customer": {...},
    "dictionary": {...}
  }
}
```

Trigger element (under *context*) can be:

- PNR: the customer event is based on a PNR event
- API: the customer event is based on a T-DNA API event
- ...

Process element (under *context*) can be:

- CustomerCreate: a new T-DNA customer is created
- CustomerUpdate: an existing T-DNA customer is updated
- CustomerDelete: a T-DNA customer is deleted

Please note that for the Batch, each line corresponds to the latest update since last batch (for instance if a profile was created and then updated, only the update will be sent, containing all profile data).

Timestamp element (under *context*) indicates when the profile has been created, updated or deleted.

amadeus

Operation element (under *activities*) can be:

- ADD: a source is added to a T-DNA customer
- UPDATE: an existing source is updated within a T-DNA customer
- DELETE: a source is removed from a T-DNA customer

4.3.2 Dictionary part and source id handling

The dictionary part of the feed contains all the references to the aggregated sources that build the profile in the customer section.

Example:

```
{
  "LiveFeed": {
    "context": {...},
    "customer": {...},
    "dictionary": {
      "derivedData": [
        {
          "id": "CEM_DERIVED_DATA-2",
          "sourceType": "CEM_DERIVED_DATA",
          "generatedAt": "2022-02-11T18:22:26Z"
        }
      ],
      "pnrSources": [
        {
          "id": "PNR-1",
          "sourceType": "PNR",
          "booking": {
            "locator": {
              "internal": {
                "value": "LC74RW"
              }
            },
            "numberInParty": 1,
            "createdAt": "2022-02-11T10:27:00Z",
            "lastModifiedAt": "2022-02-11T18:22:26Z",
            "creatorPointOfSale": {
              "officeId": "NCE0E01ME",
              "userSign": "9999WS"
            }
          },
          "envelop": 14,
          "passengerTattoo": 2,
          "version": 14
        }
      ],
      "loyaltyProfileSources": [
```

```

    {
      "id": "LOYALTY_PROFILE-1",
      "sourceType": "LOYALTY_PROFILE",
      "receivedAt": "2022-02-11T18:01:24Z",
      "locator": {
        "internal": {
          "value": "12345678"
          "context": "CSX"
          "company": {
            "code": "0E"
          }
        }
      },
      "version": 1
    },
    {
      "id": "AIR_TICKET-1",
      "sourceType": "AIR_TICKET",
      "lastModifiedAt": "2022-03-07T18:36:42Z",
      "ticket": {
        "number": "9562488486965",
        "owner": "0E",
        "issuedAt": "2022-02-25T00:00:00Z"
      },
      "envelope": 1
    }
  ],
  "ticketingSources": [{
    "id": "AIR_TICKET-1",
    "sourceType": "AIR_TICKET",
    "lastModifiedAt": "2022-03-07T18:36:42Z",
    "ticket": {
      "number": "9562488486965",
      "owner": "0E",
      "issuedAt": "2022-02-25T00:00:00Z"
    },
    "envelope": 1
  }],
  "loyaltyProfileSources": [{
    "id": "LOYALTY_PROFILE-1",
    "sourceType": "LOYALTY_PROFILE",
    "receivedAt": "2022-02-11T18:01:24Z",
    "locator": {
      "internal": {
        "value": "12345678",
        "context": "CSX",
        "company": {
          "code": "0E"
        }
      }
    },
    "version": 1
  }],
  "derivedData": []
}

```

Several source types can be present in a given profile:

- **pnrSources**: data captured from a given PNR
- **ticketingSources**: ticket information from a given PNR
- **loyaltyProfileSources**: loyalty data captured via Amadeus CSX
- **derivedData**: data generated by T-DNA (“golden profile” and metrics)
- etc.

Inside the feed, all sources are given a local identifier (e.g., PNR-1, LOYALTY-PROFILE-2...), depending on their type. The dictionary part enables to retrieve the actual identifier pertaining to each source.

For pnrSources, the record locator is in *booking/locator/internal/value*.

For `loyaltyProfileSources`, the ID is in *locator/internal/value*.

For `ticketingSources`, the ticket number is in *ticket/number*.

4.3.3 Customer part

For each data of the *customer/profile* part, there is a reference to the source the data is coming from.

Example: The flight here is coming from PNR-1:

```
{
  "LiveFeed": {
    "context": {...},
    "customer": {
      "profile": {
        "products": {
          "trips": [{
            "journeys": [{
              "origin": {
                "code": "NCE",
                "at": "2022-06-23T20:10:00+03:00"
              },
              "destination": {
                "code": "MUC",
                "at": "2022-06-23T21:00:00+03:00"
              },
              "relatedFlights": [{
                "productType": "FlightSegment",
                "tattoo": "1",
                "uuid": "NCE-MUC"
              }]
            },
            "sources": [{
              "id": "PNR-1",
              "sourceType": "PNR"
            }]
          }]
        }
      }
    },
    "dictionary": {...}
  }
}
```

Then, the *dictionary* part enables to determine the actual record locator of PNR-1.

4.4 Feed content

For the update feed (Batch or Live feed): Once an activity is detected on a profile, T-DNA executes a process comparing the old and new customer views.

If there is no difference, no update is sent.

In all cases, the full new image of the customer is pushed. This means that all sections and all sources are in the customer feed, even if the section or the source was not the updated one.

4.4.1 Profile filtering

The profile filtering aims to reduce the size of T-DNA profiles by applying filters, in order to return only relevant content to the airlines. Therefore, this feature is applicable for small, medium and large profiles.

This feature aims to manage the most relevant customer data in the context of very frequent travelers that contain several hundreds of bookings on their profiles.

The Profile Filtering Module will filter the profiles to keep only useful data and ensure the profile can be sent through the network.

There are several kinds of filters that are described below such as:

- **Source type filtering:** filter some kinds of sources that are not relevant for the consumers, for instance eCommerce, Cookies, etc.
- **Empty PNRs filtering:** The aim is to filter PNRs that do not contain relevant information as they usually do not contain any flight, service details, nor any links to other sources such as a ticket number, nor any disruption information (the definition of “empty” being defined by Amadeus)
- **Historical Data filtering:** The aim is to filter the oldest data to keep the most recent one

These filters can be configured for each airline and can be configured to be applied when the system returns the profile.

More information on this feature can be found in the “Large profile filtering” dedicated documentation.

5 Feed distribution channels

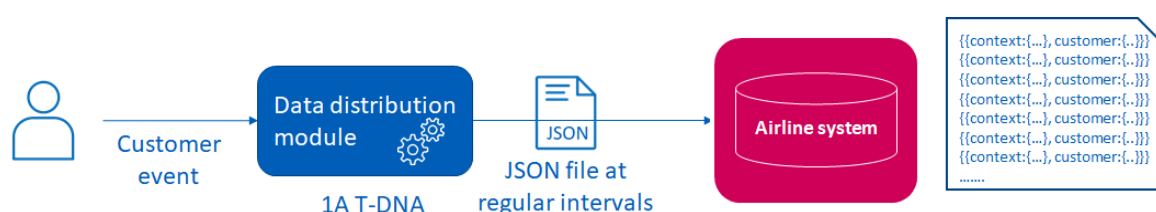
The JSON containing the updated customer data is serialized and can be distributed to the airline system either as part of a bundle (Batch) or via an online message (Live feed).

5.1 Batch (included in baseline)

All customers' profiles that were modified during a pre-set time (**daily** or weekly) and previously stored in T-DNA database are bundled into a file and sent over to the airline owner, via SFTP.

Limitations:

- Only the latest image of the customer profile since previously sent update is pushed (no history is sent)



5.2 Live feed (Add-on only for customers implemented before 2023)

Modifications in customer profile are processed and instantly sent via MQ message.

Please note that this is a near real-time feed.



For customers implemented starting 2023, the alternative of the Live feed is the T-DNA feed to DIH Data Hub.

6 Feed consumption guidelines

6.1 Profile dynamicity

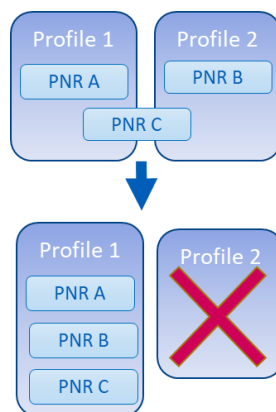
Important warning: The internal Amadeus identifier of the profile is the T-DNA customer id. We recommend the airline **not** to use this identifier as it 'volatile' and subject to change.

As mentioned above, a profile provides a view a of all records belonging to a customer at a given time. Hence, **this is not recommended to rely on the profile as a whole**, but rather as a mapping table between sources, in which additional value is provided.

As a basic use case, when the source (PNR, CSX, ...) is created or updated, the process tries to identify an already existing profile, and links it to this source. If no profile is matching, then a new one is created. Then, some more complex use cases can happen, showing the dynamicity of the profile view.

6.1.1 Merge

When a source is created/updated, and the system tries to identify a matching profile, this is possible to have two profiles matching the source. In that case, and in some conditions, the process understands the two existing profiles belong to the same individual, and they are merged into one unique profile.



In this example, PNR C allows the merge of Profile 1 and 2

Batch Feed content:

With the assumption this merge is the last action on the profiles before the Batch feed generation, then, the Batch Feed contains:

a) For profile 1, this is an addition of PNR B and C:

```
{
  "context": {
    "trigger": "PNR",
    "process": "CustomerUpdate",
    "timestamp": "2022-10-09T14:10:06Z"
  },
  "customer": {
    "profile": {
      "id": "PROFILE00000000000000000001",
      "identity": {...},
      "contact": {...},
      "products": {...},
      "metrics": [...]
    }
  },
  "dictionary": {
    "derivedData": [...],
    "pnrSources": [{
      "id": "PNR-1",
      ...
    }, {
      "id": "PNR-2",
      ...
    }, {
      "id": "PNR-3",
      ...
    }
  ]
}
```

b) for profile 2, PNR B is removed, and profile is now empty:

```
{
  "context": {
    "trigger": "PNR",
    "process": "CustomerDelete",
    "timestamp": "2022-10-09T14:10:06Z"
  },
  "customer": {
    "profile": {
      "id": " PROFILE00000000000000000002"
```

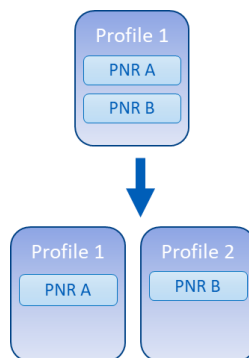
```

    }
  }
}

```

6.1.2 Split

When a source is updated, some new elements (like Passport, or Frequent Flyer) can allow the system to understand that a previously identification was not accurate enough. Hence a new profile is created.



In this example, update of PNR B implies a split of profile 1 into two different profiles

Batch Feed content

With the assumption this split is the last action on the profiles before the Batch feed generation, then, the Batch Feed contains:

a) For profile 1, the PNR B is removed, the profile contains now only PNR A:

```

{
  "context": {
    "trigger": "PNR",
    "process": "CustomerUpdate",
    "timestamp": "2022-10-12T09:21:32Z"
  },
  "customer": {
    "profile": {
      "id": "PROFILE00000000000000000001",
      ...
    }
  },
  "dictionary": {
    "derivedData": [...],
    "pnrSources": [
      {

```

```

        "id": "PNR-1",
        ...
      }
    ]
  }
}

```

b) Profile 2 is created with PNR B:

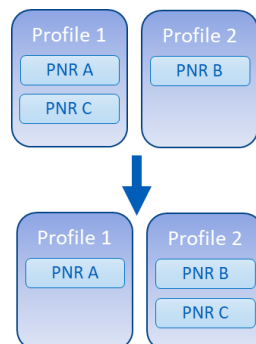
```

{
  "context": {
    "trigger": "PNR",
    "process": "CustomerCreate",
    "timestamp": "2022-10-12T09:21:32Z"
  },
  "customer": {
    "profile": {
      "id": "PROFILE00000000000000000002",
      {...}
    },
  },
  "dictionary": {
    "derivedData": [...],
    "pnrSources": [{
      "id": "PNR-2",
      ...
    }
  ]
}
}

```

6.1.3 Rehook

When a source is updated, some new elements (like Passport, or Frequent Flyer) can allow the system to understand that a previously identification can be enhanced. Then, the source is moved to another profile.



In this example, update of PNR C allows to move it from Profile 1 to Profile 2

Batch Feed content

With the assumption this split is the last action on the profiles before the Batch feed generation, then, the Batch Feed contains:

a) For profile 1, the PNR C is removed, the profile contains now only PNR A:

```
{
  "context": {
    "trigger": "PNR",
    "process": "CustomerUpdate",
    "timestamp": "2022-10-12T09:45:12Z"
  },
  "customer": {
    "profile": {
      "id": "PROFILE00000000000000000001",
      ...
    }
  },
  "dictionary": {
    "derivedData": [],
    "pnrSources": [
      {
        "id": "PNR-1",
        ...
      }
    ]
  }
}
```

b) Profile 2 now contains PNR B and C:

```
{
  "context": {
    "trigger": "PNR",
    "process": "CustomerUpdate",
    "timestamp": "2022-10-12T09:45:12Z"
  },
  "customer": {
    "profile": {
      "id": "PROFILE00000000000000000002",
      ...
    }
  },
  "dictionary": {
    "derivedData": [...],
    "pnrSources": [
      {
        "id": "PNR-2",

```

```

    ...
  },
  {
    "id": "PNR-3",
    ...
  }
]
}

```

6.2 Profile identifier

The *customer* object contains an identifier of the described profile (in *customer.profile.id*). As explained in previous part, the profile is dynamic, and the same individual can be moved from one profile to another. Hence, the profile identifier must be used with extreme cautious, and never be strongly attached to an individual.

Here is a list of points of attention when using the profile identifier:

- This id is not shared across 1A environments (Prod, UAT, MIG, ...)
- It must not be used as primary key in the client system, as both databases (T-DNA and client) can pursue different needs
- It must not be spread to downstream applications, as synchronization with T-DNA database could be heavy to put in place

6.3 Which data to record?

6.3.1 Source references

Airline's system can store sources (PNR, Loyalty, ...) coming from another channel than T-DNA. In that case, here is the recommendation to retrieve them in the T-DNA feed.

6.3.1.1 PNR

PNRs references are described in the object *dictionary.pnrSources*, containing all PNRs belonging to the profile.

Each PNR is known in a unique manner with a triplet Record Locator, creation date, passenger tattoo:

- **Record Locator:** *booking.locator.internal.value*
- **Creation date:** *booking.createAt*

- Passenger tattoo: *passengerTattoo*

Example:

```
{
  "LiveFeed": {
    "context": {...},
    "customer": {...},
    "dictionary": {
      "pnrSources": [
        {
          "id": "PNR-3",
          "sourceType": "PNR",
          "passengerTattoo": 2,
          "envelop": 1,
          "booking": {
            "locator": {
              "internal": {
                "value": "O5NOV8"
              }
            },
            "numberInParty": 1,
            "createdAt": "2021-09-08T10:50:00Z",
            "lastModifiedAt": "2022-07-08T11:41:02Z",
            "creatorPointOfSale": {
              "officeId": "NCE0E01ME",
              "userSign": "0001AA"
            }
          },
          "version": 1
        },
        {...}
      ]
    }
  }
}
```

6.3.1.2 Ticket

Tickets references are described in the object *dictionary.ticketingSources*, which is a table containing all tickets (e-tickets and EMDs) belonging to the profile.

Each ticket is known in a unique manner with a duet Document number, creation date:

- Document number: *ticket.number*
- Creation date: *ticket.issuedAt*

Example:

```
{
  "LiveFeed": {
    "context": {...},
    "customer": {...}
  },
  "dictionary": {
    "ticketingSources": [
      {
        "lastModifiedAt": "2022-08-15T00:23:10Z",
        "id": "AIR_TICKET-11",
        "sourceType": "AIR_TICKET",
        "ticket": {
          "number": "9569853091443",
          "owner": "0E",
          "issuedAt": "2022-04-15T00:00:00Z"
        },
        "envelope": 5,
        "creatorPointOfSale": {
          "userSign": "E"
        }
      },
      {...}
    ]
  }
}
```

6.3.1.3 CSX (loyalty profiles)

CSX (loyalty profiles) references are described in the object *dictionary.loyaltyProfileSource*.

The Frequent Traveller is known with the duet Frequent Flyer number and program owner:

- Frequent Flyer number: *locator.internal.value*
- Program owner: *locator.internal.company.code*

Example:

```
{
  "LiveFeed": {
    "context": {...},
    "customer": {...},
    "dictionary": {
      "loyaltyProfileSource": [{
        "receivedAt": "2022-01-14T11:05:21Z",
        "id": "LOYALTY_PROFILE-1",

```

```

        "sourceType": "LOYALTY_PROFILE",
        "locator": {
            "internal": {
                "value": "12345678",
                "context": "CSX",
                "company": {
                    "code": "0E"
                }
            }
        },
        "version": "2"
    ]
}

```

6.3.2 Golden profile

Regarding the *identity* (names, demographics), *contact*, *memberships* and *metrics*, the airline can rely on the derived data computed by T-DNA (i.e., “golden profile”) – or look for data against a specific source reference (PNR, TKT, etc.).

In order to record this data, it is necessary to:

1. Check the dictionary in order to find out the local identifier of the T-DNA derived data

This info is the *id* of the CEM_DERIVED_DATA *sourcetype* inside the *derivedData* part of the *dictionary*.

Example: Here the id is CEM_DERIVED_DATA-2:

```

{
  "LiveFeed": {
    "context": {...},
    "customer": {...}
  },
  "dictionary": {
    "derivedData": [
      {
        "id": "CEM_DERIVED_DATA-2",
        "sourceType": "CEM_DERIVED_DATA",
        "generatedAt": "2022-02-12T08:27:28Z"
      }
    ]
  }
}

```

2. Inside each entity in *identity*, *contact*, *memberships* and *metrics*, retrieve the one for which the source has the identifier previously retrieved

Example: Here regarding the *firstName* and *lastName*:

```
{
  "LiveFeed": {
    "context": {...},
    "customer": {
      "profile": {
        "id": "C538784795",
        "identity": {
          "names": [
            {
              "universal": {
                "firstName": "PETER",
                "lastName": "DOE",
                "sources": {
                  "id": "PNR-1",
                  "sourceType": "PNR"
                }
              }
            }
          ],
          {
            "universal": {
              "firstName": "PETER",
              "lastName": "DOE",
              "rank": "1",
              "sources": {
                "id": "CEM_DERIVED_DATA-2",
                "sourceType": "CEM_DERIVED_DATA"
              }
            }
          }
        ]
      },
      "metrics": [
        {
          "name": "Total Revenue",
          "code": "TRV",
          "value": "83.79",
          "valueFormat": "FLOAT",
          "sources": [
            {
              "id": "CEM_DERIVED_DATA-2",
              "sourceType": "CEM_DERIVED_DATA"
            }
          ]
        }
      ]
    }
  }
}
```

```

    ]
  }
]
},
"dictionary": {
  "derivedData": [
    {
      "id": "CEM_DERIVED_DATA-2",
      "sourceType": "CEM_DERIVED_DATA",
      "generatedAt": "2022-02-12T08:27:28Z"
    }
  ],
  "pnrSources": [
    {
      "id": "PNR-1",
      "sourceType": "PNR",
      "booking": {
        "locator": {
          "internal": {
            "value": "QSNYGG"
          }
        }
      },
      "numberInParty": "1",
      "createdAt": "2022-02-01T16:01:00Z",
      "lastModifiedAt": "2022-02-01T16:01:53Z",
      "creatorPointOfSale": {
        "officeId": "NCE0E01ME",
        "userSign": "0001AA"
      }
    },
    {
      "envelop": "0",
      "passengerTattoo": "1",
      "version": "0"
    }
  ]
}
}
}
}

```

6.3.3 Origin and Destination

For each itinerary booked by the traveller, the origin and destination are computed by the system. It is described in object *customer.profile.products.trips.journeys* with:

- The itinerary origin airport code:
customer.profile.products.trips.journeys.origin.code

- The itinerary destination airport code:
customer.profile.products.trips.journeys.destination.code

This itinerary is linked to a PNR, which can be retrieved with the corresponding source id described in the object *customer.profile.products.trips.sources.id*. Then this id is referenced in the object *dictionary.pnrSources*, containing the triplet record locator, creation date, passenger tattoo.

Example:

```
{
  "LiveFeed": {
    "context": {...},
    "customer": {
      "profile": {
        "products": {
          "trips": [{
            "journeys": [{
              "origin": {
                "code": "NCE",
                "at": "2022-06-23T20:10:00+03:00"
              },
              "destination": {
                "code": "MUC",
                "at": "2022-06-23T21:00:00+03:00"
              },
              "relatedFlights": {
                "productType": "FlightSegment",
                "tattoo": "1",
                "uuid": "NCE-MUC"
              }
            }
          ]
        },
        "sources": [{
          "id": "PNR-2",
          "sourceType": "PNR"
        }]
      }
    }
  },
  "dictionary": {
    "pnrSources": {
      "id": "PNR-2",
      "sourceType": "PNR",
      "passengerTattoo": "5",
      "envelop": "32",
      "booking": {
        "locator": {
          "internal": {
```

```

        "value": "RBIAGW"
      },
      "numberInParty": "5",
      "createdAt": "2022-06-21T03:03:00Z",
      "lastModifiedAt": "2022-06-23T17:12:35Z",
      "creatorPointOfSale": {
        "officeId": "NCE0E01ME",
        "userSign": "0001AA"
      },
      "version": "32"
    },
  },
}

```

6.3.4 Metrics

For each profile, when possible, T-DNA calculates a set of metrics, enhancing the knowledge on this customer. All metrics are available in an array located in *customer.profile.metrics*.

Each metric is defined as a key value. When searching for a specific metric, it is needed to loop on all the metrics, and retrieve the specific code, in *customer.profile.metrics[i].code*. Then, the value of the metric is in *customer.profile.metrics[i].value*.

For [example](#), for profiles containing at least one PNR, to easily know the last booking date, it is possible to check the value of the metric whose code is LBD:

```

{
  "context": {...},
  "customer": {
    "profile": {
      ...
      "metrics": [{
        "name": "Average Ancillary Services Revenue",
        "code": "AASR",
        "value": "0",
        "valueFormat": "FLOAT",
        "sources": [{
          "id": "CEM_DERIVED_DATA-3",
          "sourceType": "CEM_DERIVED_DATA"
        }]
      },
      {
        "name": "Last Booking Date",
        "code": "LBD",

```

```

    "value": "2022-07-18T07:45:00Z",
    "valueFormat": "DATE_TIME",
    "sources": [{
      "id": "CEM_DERIVED_DATA-3",
      "sourceType": "CEM_DERIVED_DATA"
    }]
  }
]
}
}
}

```

6.3.5 Making sure to record the latest version

A **version** attribute is added in the grammar of profiles with string as value. It follows the pattern of: {major}.{minor}, for example, 3.4 correspond to major = 3 and minor = 4.

The version contains 2 parts:

- **Major**: Incremented by 1 each time if links on the profile have changed
 - o For example, a new source is added to the profile, a source was moved to another profile, etc.
- **Minor**: Incremented by 1 each time if any update of any source of the profile has changed and will be re-initialized to 0 when major version changed.
 - o For example, an update on the PNR that doesn't change the linkage.

To compare 2 versions, external systems can firstly compare the major version, if the message contains a higher major version, no matter what the minor version is, this message is more recent than the other. But if both messages have the same major version, we compare the minor version, and the message with higher minor version is more recent than the other.

For example, if the current version for a profile is 1.1 which correspond to (Major: 1, Minor: 1), and when there is an update on one of the sources in the profile, but no sources are added or removed from the profile in consequence of this update, only the minor version is incremented, and the version become: 1.2(Major: 1, Minor: 2). But if the update of the source triggered the adding of a new source to this profile, major version is incremented while minor version is re-initialized to 0, and the version become 2.0 (Major: 2, Minor: 0).

In case of reception of multiple update feed messages of the same customer, **the external system should use the *version* attribute to determine which message represents the latest event.**

Example: Message 1:

```

{
  "LiveFeed": {
    "context": {
      "trigger": "PNR",

```

```

    "process": "CustomerUpdate",
    "timestamp": "2022-02-11T18:22:27Z",
    "activities": [
      {
        "operation": "ADD",
        "source": {
          "id": "PNR-1",
          "sourceType": "PNR"
        }
      }
    ],
    "customer": {
      "profile": {
        "id": "C123",
        "version": "2.3",
        ...
      }
    },
    "dictionary": {...}
  }
}

```

Example: Message 2:

```

{
  "LiveFeed": {
    "context": {
      "trigger": "LOY",
      "process": "CustomerUpdate",
      "timestamp": "2022-02-11T18:22:27Z",
      "activities": [
        {
          "operation": "ADD",
          "source": {
            "id": "LOY-1",
            "sourceType": "LOY"
          }
        }
      ]
    },
    "customer": {
      "profile": {
        "id": "C123",
        "version": "1.2",
        ...
      }
    }
  },
  ...
}

```



```
    "dictionary": {...}  
  }  
}
```

Two messages above are generated following the events for the same customer C123, and at the same moment: 2022-02-11T18:22:27Z. The first message is the event of adding a PNR source into the customer, and the second message is representing the adding of loyalty source into the same customer.

The external system can rely on the *version* attribute at customer/profile/version level to understand the sequence of different events for the same customer without decoding sources.

In the example above, as the *version* in the message 1 is 2.3 ("major": 2, "minor": 3) which is higher than the *version* 1.2("major": 1, "minor": 2) in the message 2, in consequence, event generating the message 1 is more recent than the event for message 2.

For airlines for which the version field was added after some profiles were already created, the external system should consider that for a given profile, an event with any version number is more recent than a message without a version number.

7 Appendix

This [Batch_LiveFeed_V3_0.xsd](#) is available in Amadeus Service Hub [here](#), and is subject to evolve in a backward compatible way. Airline system has to be able to support this kind of enhancements.

The associated [BatchExample_V3_0.txt](#) of a Batch file (the LiveFeed being the same but containing only one 'line' per transaction) is available in Amadeus Service Hub [here](#).

8 Glossary

Term	Description
CEM	Customer Experience Management, Amadeus product – former name for T-DNA
CI	Customer Image. T-DNA internal storage format. Each source of data is stored in T-DNA in a CI form.
T-DNA	Traveler DNA, new name for Customer Experience Management (CEM), Amadeus product